AGENTIC AI ARCHITECTURE

Intelligent Retrieval, Reasoning & Action

Ball Corporation — Enterprise Data to AI Agent Blueprint

SAP · JD Edwards · FactoryTalk → Databricks Lakehouse → Agentic AI

Detailed Component Architecture & Implementation Guide

Confidential · May 2026

TABLE OF CONTENTS

1. ARCHITECTURE OVERVIEW — THE SEVEN LAYERS

Ball Corporation's Agentic AI Architecture implements intelligent retrieval, reasoning, and action capabilities by connecting enterprise ERP data (SAP, JD Edwards, FactoryTalk) through a governed Databricks Lakehouse to AI agents that can answer questions, analyze data, and safely execute actions against production systems.

The architecture consists of seven functional layers, two cross-cutting concerns (governance and platform), and delivers value through three interaction modes: RAG retrieval, Text-to-SQL structured queries, and Function Calling for safe SAP actions.

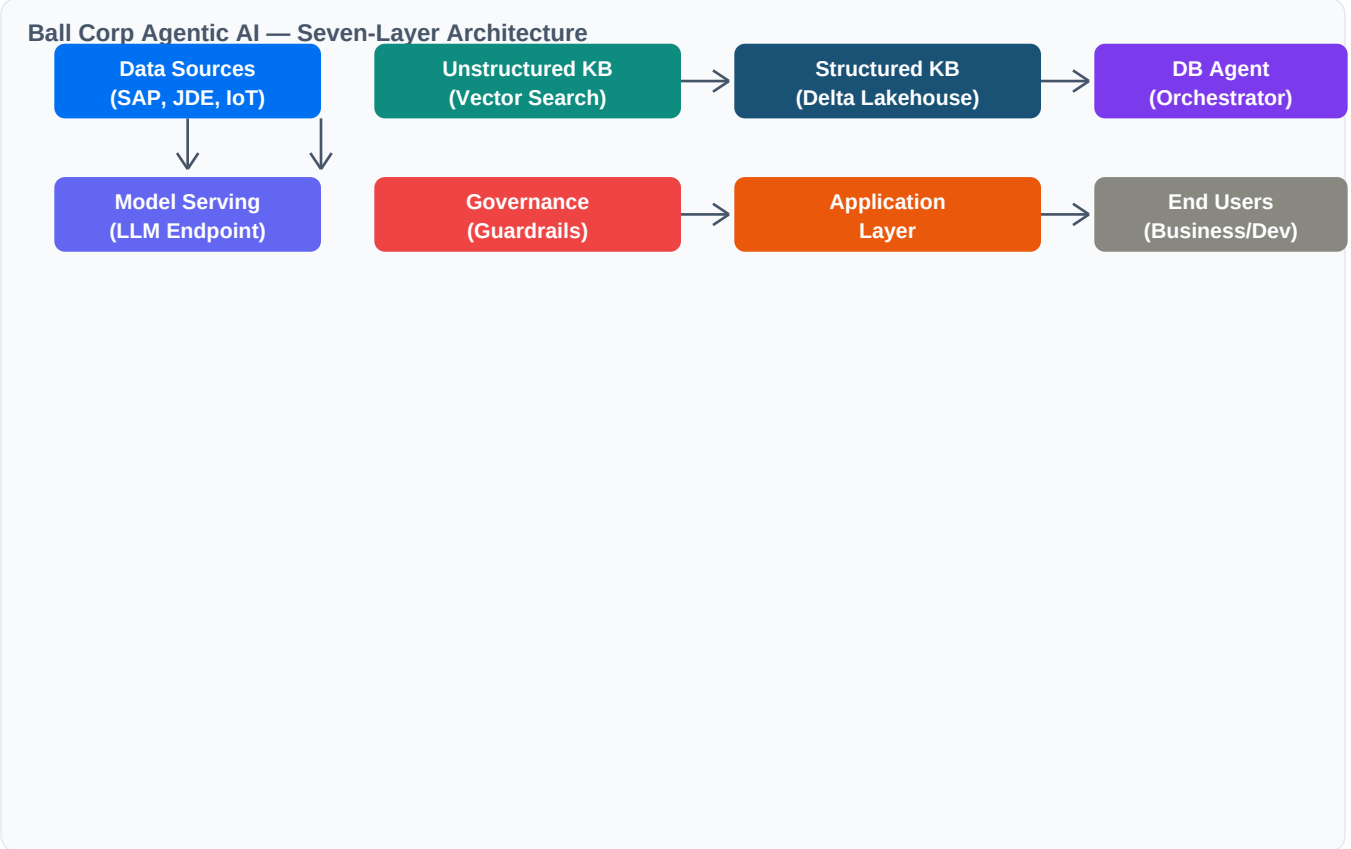

Figure 1.1 — Architecture Overview

Layer	Component	Ball Corp Implementation
1. Data Sources	SAP, JDE, FactoryTalk, Documents, APIs	10 SAP modules via CDS/ODP, JDE via JDBC, FactoryTalk via MQTT/Kafka, SOPs/PDFs
2. Unstructured KB	Vector Search Index	Documents chunked → embedded → stored in Databricks Vector Search for RAG
3. Structured KB	Delta Lakehouse	Medallion (Bronze→Silver→Gold) in Unity Catalog with semantic YAML manifests
4. DB Agent	AI Orchestrator	Understands query, retrieves context, executes tools, generates response
5. Model Serving	LLM Endpoint	GPT-4o / DBRX / Llama hosted on Databricks Model Serving (private)

6. Governance	Guardrails + Registry	Tool registry, execution limits, Databricks + APIM dual-layer safety
7. Application	End User Interface	Databricks App, Power BI, REST APIs, custom applications

Figure 1.2 — Seven-Layer Component Mapping

2. LAYER 1: DATA SOURCES

Ball Corporation's data sources span three enterprise systems (SAP, JD Edwards, FactoryTalk) plus unstructured documents, APIs, and streaming data. The extraction strategy uses CDS Views from SAP to capture business logic directly — not just raw tables.

2.1 Source System Inventory

Source	Type	Modules / Data	Extraction Method
SAP ECC/S4	ERP (Structured)	FI, CO, SD, MM, PP, IBP, QM, PS, PM, SCM	CDS Views via ODP/CDC, SAP BDC Delta Sharing
JD Edwards	ERP (Structured)	Finance, Distribution, Manufacturing, Procurement	JDBC/ODBC batch, Trigger-based CDC
FactoryTalk	IoT (Streaming)	Sensor telemetry, OEE, PLC, alarms, historian	MQTT/OPC-UA → Event Hub → Auto Loader
Documents	Unstructured	SOPs, defect reports, manuals, specifications	PDF/text extraction, chunking → embeddings
APIs/SaaS	Semi-structured	Weather, market, supplier data feeds	REST API → ADF → Bronze layer

Figure 2.1 — Data Source Inventory

2.2 Why CDS Views (Not Raw Tables)

CDS Views encapsulate SAP business logic at the application layer. They deliver pre-joined, pre-calculated, and authorization-checked payloads so the AI agent receives business-ready data instead of cryptic German field names:

- **Joins:** Pre-built joins — header + item tables already flattened (no VBAK/VBAP manual joins)
- **Calculations:** Calculated fields — net values with currency conversion, material statuses resolved
- **Security:** Authorization parameters — row-level security applied before data leaves SAP
- **Naming:** Business-friendly names — VBELN becomes sales_document_number, NETWR becomes net_value

3. LAYER 2: UNSTRUCTURED KNOWLEDGE BASE

The Unstructured Knowledge Base stores documents, SOPs, defect reports, and manuals as vector embeddings in Databricks Vector Search. When the AI agent needs context that doesn't exist in structured tables — like 'What does our SOP say about cleaning procedure for Line 7?' — it retrieves relevant chunks via semantic similarity search.

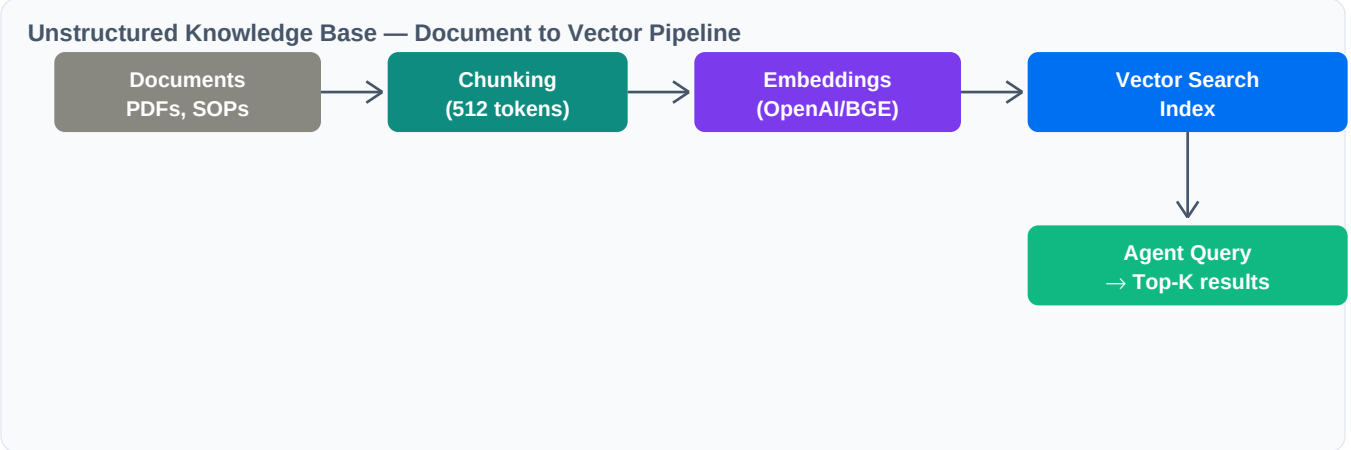

Figure 3.1 — Vector Search Pipeline

3.1 Pipeline Steps

- **Ingest:** Load PDFs, Word docs, and text files from ADLS Gen2 or SharePoint into Databricks volumes
- **Chunk:** Split documents into overlapping chunks (512 tokens with 50-token overlap) preserving section boundaries
- **Embed:** Generate dense vector embeddings using OpenAI text-embedding-3-small or open-source BGE-large model
- **Index:** Store embeddings in a Databricks Vector Search index with metadata (source_file, page_number, section)
- **Retrieve:** At query time, agent sends natural language question → top-K similar chunks returned as LLM context

3.2 Ball Corp Document Types

Document Type	Content	Agent Use Case
Manufacturing SOPs	Equipment procedures, cleaning protocols	Answer process questions, check compliance steps
Quality Defect Reports	Inspection results, root cause analysis, CAPA	Defect pattern analysis, preventive recommendations
Maintenance Manuals	Equipment specs, PM schedules, spare parts	Troubleshooting assistance, parts lookup
Safety Data Sheets	Chemical handling, emergency procedures	Safety protocol queries, compliance verification
SAP Configuration Docs	System settings, custom ABAP rules, exit logic	Agent understands custom business rules

Figure 3.2 — Document Types for Vector Knowledge Base

4. LAYER 3: STRUCTURED KNOWLEDGE BASE

The Structured Knowledge Base is the Databricks Medallion Lakehouse storing all ERP transactional and master data in Delta Lake format, governed by Unity Catalog. The agent queries this via Text-to-SQL — reading semantic YAML manifests to understand table relationships before generating SQL.

4.1 Medallion Architecture

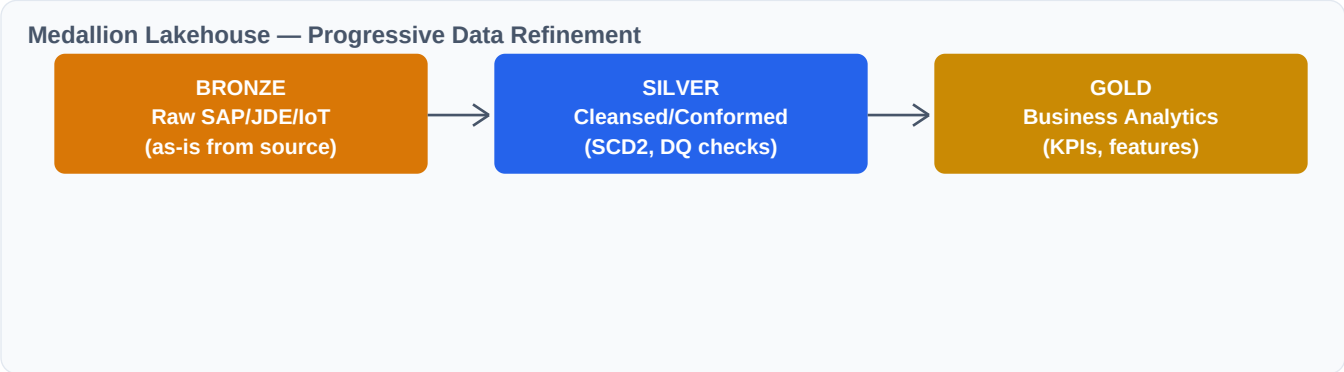

Figure 4.1 — Medallion Architecture

4.2 Unity Catalog — Data Mesh Domains

Data is organized into domain-specific catalogs following the Data Mesh pattern. Each domain team owns their catalog and publishes stable data products with schema contracts:

Domain Catalog	Owner	Key Tables	Agent Query Examples
sap_domain_finance	Corporate Finance	fi_gl_transactions, co_cost_centers	Revenue by region, cost center analysis
sap_domain_sales	Commercial Ops	sd_orders, sd_customers, sd_billing	Open orders > $50K, customer credit status
sap_domain_logistics	Supply Chain	mm_materials, pp_production_orders	Inventory levels, BOM explosion, vendor perf
sap_domain_quality	Quality Team	qm_inspections, pm_work_orders	Defect rates by plant, maintenance backlog

Figure 4.2 — Unity Catalog Domain Catalogs

4.3 Semantic YAML Manifests

Each domain publishes YAML schema manifests that the AI agent reads before writing SQL. These describe table relationships, join conditions, and business context so the LLM generates accurate queries:

```
table_name: sap_domain_sales.sales_products.global_sd_orders
description: "All sales order items and headers across NA and EU."
primary_key: "sales_document_id"
joins:
  - target: "global_sd_customers"
    on: "customer_id = global_customer_id"
    description: "Filter orders by customer attributes"
business_rules:
  - "NETWR is already converted to reporting currency"
  - "Status 'C' = Fully Processed, 'A' = Open"
```

5. LAYER 4: DB AGENT — THE AI ORCHESTRATOR

The DB Agent is the central orchestrator that receives user queries, decides the best retrieval strategy, executes tools, and generates responses. It operates as a Databricks AI Agent using the Mosaic AI Agent Framework, connecting to both knowledge bases and the tool registry.

5.1 Agent Capabilities

Capability	How It Works	Ball Corp Example
Understands user query	NLU parsing to classify intent: question vs. action vs. analysis	User: "Show me all open orders for Acme Corp over $50K"
Plans and decides	Determines retrieval strategy: RAG, Text-to-SQL, or Tool Call	Agent decides: Text-to-SQL (structured data query)
Retrieves context	Pulls from Vector Search (unstructured) or UC (structured)	Reads YAML manifest, generates SQL, queries Gold tables
Executes actions	Invokes registered tools via function calling + guardrails	Calls clear_credit_block tool through APIM → SAP OData
Generates response	Synthesizes retrieved data into natural language answer	Returns formatted table + insight: "3 open orders totaling $142K"

Figure 5.1 — DB Agent Capabilities

5.2 Three Interaction Modes

Mode	Trigger	Data Path	Example
RAG Retrieval	Context/knowledge question	Vector Search → Top-K chunks → LLM	What does our SOP say about cleaning Line 7?
Text-to-SQL Query	Structured data question	YAML manifest → SQL → Gold tables	Show all open POs with delivery delays
Function Calling	Action request (write/update)	Tool Registry → Guardrails → APIM	Release blocked PO #4500012345

Figure 5.2 — Agent Interaction Modes

6. LAYER 5: MODEL SERVING LAYER

The Model Serving Layer hosts the LLM / Foundation Model that powers the agent's reasoning capabilities. Models are deployed on Databricks Model Serving endpoints with private networking, auto-scaling, and A/B testing support.

Model	Provider	Use Case	Deployment
GPT-4o	Azure OpenAI	Complex reasoning, multi-step planning, action generation	External model endpoint (Azure private link)
DBRX	Databricks	General-purpose queries, cost-optimized workloads	Databricks Model Serving (provisioned throughput)
Llama 3.1 70B	Meta (open)	On-premise sensitive data processing, fine-tuned tasks	Databricks Model Serving (GPU clusters)
BGE-large	BAAI (open)	Document embedding for Vector Search	Databricks Model Serving (CPU endpoint)

Figure 6.1 — Model Serving Configuration

- **Private Endpoints:** All model endpoints deployed with private networking (VPC/Private Link) — no public internet exposure
- **Auto-Scaling:** Auto-scaling from 0 to N instances based on request throughput with provisioned throughput for latency-sensitive workloads
- **Model Registry:** MLflow model registry for versioning, A/B testing between model versions, and automated rollback

7. LAYER 6: AGENT GOVERNANCE & GUARDRAILS

The governance layer implements the semantic memory, tool registry, and dual-layer execution guardrails that ensure the agent acts safely and accurately. No action reaches SAP production without passing through both Databricks policy validation and Azure APIM gateway checks.

7.1 Semantic Memory Layer

- **Schema Manifests:** Schema manifests (YAML) stored in sap_enterprise.agent_governance.schema_manifests — table relationships, join conditions, business rules for Text-to-SQL
- **Business Glossary:** Business glossary in agent_governance.business_glossary — SAP-to-English field mapping, domain context for RAG injection
- **AI Documentation:** AI-assisted metadata enrichment — Unity Catalog auto-generates column descriptions from query patterns and data profiling

7.2 Tool Registry

Registered SAP actions with precise natural language descriptions that tell the LLM exactly when and how to invoke each tool:

Tool	Description (for LLM)	Guardrail
clear_credit_block	Clear financial credit hold. Requires order_id, reason_code. Block if amount > $50K.	Supervisor approval if > $50K
create_pm_order	Create maintenance work order. Requires equipment_id, priority.	Business hours only
release_purchase_order	Approve blocked PO. Requires po_number, approver_id.	Budget threshold check
update_delivery_date	Reschedule delivery. Cannot reschedule if goods issued.	Status validation (no post-GI)

Figure 7.1 — Tool Registry with Guardrails

7.3 Dual-Layer Execution Guardrails

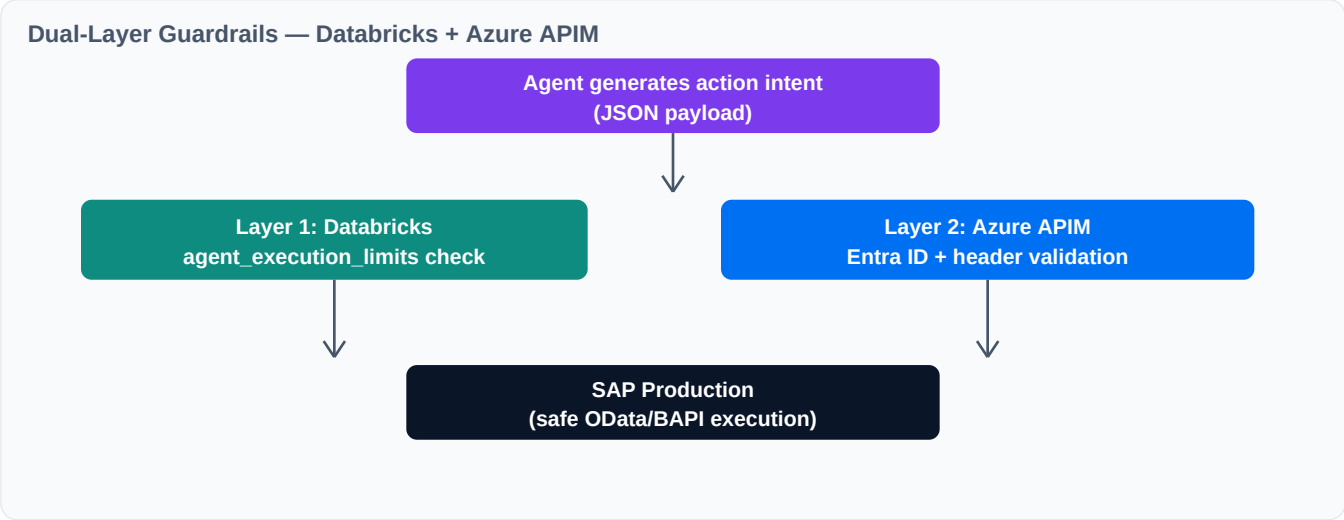

Figure 7.2 — Dual-Layer Execution Guardrails

8. LAYER 7: APPLICATION LAYER & END USERS

The application layer delivers agent capabilities to end users through multiple channels — a Databricks App chat interface for conversational interaction, Power BI dashboards for analytics, REST APIs for custom application integration, and embedded widgets for existing business tools.

Interface	Users	Capabilities
Databricks App	Business users, plant managers	Conversational chat, natural language queries, action requests
Power BI	Data analysts, executives	Dashboards connected to Gold layer, AI-generated insights
Web / API Interface	Developers, integration teams	REST API endpoints for embedding agent capabilities in custom apps
Custom Applications	Operations, maintenance	Embedded agent in existing tools (MES, CMMS, ERP portals)

Figure 8.1 — Application Interfaces

9. CROSS-CUTTING: GOVERNANCE, SECURITY & COMPLIANCE

Concern	Implementation	Tool
IAM / Access Control	Entra ID SSO, Unity Catalog GRANT/REVOKE, role-based access per domain catalog	Azure AD + UC
Data Encryption	TLS 1.3 in transit, AES-256 at rest, customer-managed keys (CMK)	Azure Key Vault
Row/Column Security	Row-level security policies, column masking functions for PII (email, SSN, bank accounts)	Unity Catalog
Audit Logging	Every agent action logged to action_audit_log with payload hash for tamper detection	UC + Log Analytics
Data Lineage	Automatic table/column-level lineage from Bronze → Silver → Gold tracked in UC	Unity Catalog
Monitoring	Model latency, token usage, error rates, guardrail trigger frequency dashboards	Databricks + Azure
Compliance	SOX 7-year retention, GDPR right to erasure, CCPA consumer transparency	Delta Lake time-travel

Figure 9.1 — Cross-Cutting Governance Matrix

10. PLATFORM FOUNDATION: DATABRICKS + AZURE

Component	Technology	Ball Corp Configuration
Lakehouse Platform	Databricks Unity Catalog + Delta Lake	Multi-catalog Data Mesh with domain ownership (finance, sales, logistics)
Cloud Storage	Azure Data Lake Storage Gen2	Hierarchical namespace, private endpoints, regional containers
Compute	Databricks Spark + Serverless SQL	Auto-scaling clusters for ETL, serverless for agent SQL queries
Networking	Azure VNet + Private Link	No public endpoints, SHIR for on-prem SAP, ExpressRoute
Secrets	Azure Key Vault + Databricks Secrets	Service principal credentials, API keys, SAP connection strings
CI/CD	Databricks Asset Bundles + Terraform	Infrastructure as code, automated pipeline deployment, GitOps

Figure 10.1 — Platform Foundation Components

11. END-TO-END DATA FLOW: QUERY TO ACTION

11.1 Example: Sales Order Query

- **User Query:** User asks: 'Show me all open sales orders for Acme Corp over $50,000'
- **Intent Classification:** Agent classifies intent as Text-to-SQL (structured data question)
- **Schema Lookup:** Agent reads YAML manifest for sap_domain_sales.sales_products.global_sd_orders
- **SQL Generation:** Agent generates SQL: SELECT * FROM global_sd_orders WHERE customer_name LIKE '%Acme%' AND net_value > 50000 AND status = 'A'
- **Query Execution:** Query executes on Databricks Serverless SQL Warehouse → returns 3 matching orders
- **Response Generation:** Agent synthesizes natural language response with formatted table and insight

11.2 Example: Action Execution (Clear Credit Block)

- **User Request:** User asks: 'Clear the credit block on order 4500012345 — customer paid invoice'
- **Intent Classification:** Agent classifies intent as Function Calling (action request)
- **Tool Matching:** Agent reads tool_registry → matches clear_credit_block tool
- **Payload Generation:** Agent generates JSON: {order_id: '4500012345', reason_code: 'PAID', user_id: 'jsmith'}
- **Guardrail Layer 1:** Layer 1: Databricks checks agent_execution_limits → order amount $32K < $50K threshold → PASS
- **Guardrail Layer 2:** Layer 2: Azure APIM validates Entra ID, maps to SAP user profile, checks HTTP headers → PASS
- **SAP Execution:** APIM translates JSON → SAP OData PATCH call → credit block cleared in SAP production
- **Audit Trail:** Action logged to action_audit_log with timestamp, agent_id, tool_name, payload_hash

12. IMPLEMENTATION ROADMAP

Phase	Timeline	Deliverables
Phase 1: Foundation	Weeks 1-4	Deploy Unity Catalog agent_governance schema Identify top CDS Views per SAP module Build YAML schema manifests for sales + finance Setup Databricks Vector Search index
Phase 2: Knowledge Bases	Weeks 5-8	Ingest SAP data via CDS Views → Medallion layers Chunk + embed SOPs and defect reports Build business glossary and semantic metadata Configure Model Serving endpoint (GPT-4o)
Phase 3: Agent Development	Weeks 9-12	Build DB Agent with Mosaic AI Agent Framework Implement RAG + Text-to-SQL interaction modes Create tool_registry with initial 5 SAP actions Deploy dual-layer guardrails (Databricks + APIM)
Phase 4: Production Rollout	Weeks 13-16	End-to-end testing with business users Expand tool registry to 20+ actions Deploy Databricks App chat interface Enable audit logging + compliance dashboards

Figure 12.1 — 16-Week Implementation Roadmap

12.1 Key Benefits

- **Unified Knowledge:** Combines structured (Delta tables) and unstructured (Vector Search) data into a unified retrieval system
- **Smart Routing:** Intelligent retrieval and reasoning — agent decides the best strategy per query (RAG, SQL, or Action)
- **Enterprise Safety:** Scalable, secure, and governed — dual-layer guardrails prevent unauthorized SAP modifications
- **Action Capability:** Actionable insights for users — not just answers, but the ability to execute actions directly from the chat
- **Enterprise Scale:** Built for enterprise AI workloads — Databricks Lakehouse + Unity Catalog + Azure AI Foundry